



**CONSIDERE SER MEMBRO DO CANAL NO YOUTUBE E GANHE
ACESSO A CONTEÚDOS EXCLUSIVOS!**

youtube.com/@ArnaldoJunior



JavaScript

O que é?

- ▶ Linguagem de programação web;
- ▶ Adiciona interatividade a uma página web;
- ▶ Permite criar aplicações web ricas.

ECMAScript

- ▶ A linguagem JavaScript é padronizada pelo comitê ECMA TC39, chamada **ECMAScript**;
- ▶ O ECMAScript define:
 - ▶ Sintaxe da linguagem;
 - ▶ Mecanismos para manuseio de erros;
 - ▶ Tipos;
 - ▶ Objetos e funções embutidos.



Mudar conteúdo HTML

Aceita aspas duplas ou simples:

```
document.getElementById("demo").innerHTML = "Hello JavaScript";
```

```
document.getElementById('demo').innerHTML = 'Hello JavaScript';
```

Mudar valores de atributos HTML

```
document.getElementById('myImage').src='pic_bulbon.gif'
```

Mudar estilo (CSS)

```
document.getElementById("demo").style.fontSize = "35px";
```

```
document.getElementById("demo").style.display = "none";
```



Declarações

Ponto e Vírgula (;)

- ▶ Separam as declarações JavaScript;
- ▶ **Adicione um ponto e vírgula ao fim de cada declaração** executável;
- ▶ Embora **não** seja **obrigatório**, é altamente recomendável;
- ▶ Permite várias declarações em uma linha.

```
a = 5; b = 6; c = a + b;
```

Espaço em Branco

- ▶ JavaScript ignora múltiplos espaços;
- ▶ Você pode adicionar espaços em branco para tornar seu **script mais legível**:

```
var person = "Hege";  
var person="Hege";
```

- ▶ É uma boa prática colocar espaços ao redor dos operadores (= + - * /):

```
var x = y + z;
```

Blocos de Código

- ▶ As declarações podem ser agrupadas em blocos de códigos;

```
function myFunction() {  
    document.getElementById("demo1").innerHTML = "Hello Dolly!";  
    document.getElementById("demo2").innerHTML = "How are you?";  
}
```

Palavras-Chave

Keyword	Description
break	Terminates a switch or a loop
continue	Jumps out of a loop and starts at the top
debugger	Stops the execution of JavaScript, and calls (if available) the debugging function
do ... while	Executes a block of statements, and repeats the block, while a condition is true
for	Marks a block of statements to be executed, as long as a condition is true
function	Declares a function
if ... else	Marks a block of statements to be executed, depending on a condition
return	Exits a function
switch	Marks a block of statements to be executed, depending on different cases
try ... catch	Implements error handling to a block of statements
var	Declares a variable



Sintaxe

Valores

- ▶ Existem dois tipos de valores: valores fixos (**literais**) e valores variáveis (**variáveis**);
- ▶ Regras para escrever literais:
 - ▶ Números são escritos com ou sem casas decimais;
 - ▶ *Strings* podem ser escritas dentro de aspas simples ou duplas.

"John Doe"

'John Doe'

Variáveis

- ▶ Usadas para armazenar valores;
- ▶ Declaradas com a palavra-chave **var**;
- ▶ Um sinal de igual (=) é usado para atribuir valores às variáveis;

```
var x;  
  
x = 6;
```

Operadores Aritméticos

► São eles:

- + (adição);
- - (subtração);
- * (multiplicação);
- / (divisão).

$$(5 + 6) * 10$$

Operador de Atribuição

- Utilizado para atribuir valores às variáveis (=);

```
var x, y;  
x = 5;  
y = 6;
```

Comentários

- ▶ Não são executados;
- ▶ Usar barras duplas `//` ou `/* */`

```
var x = 5;    // I will be executed
```

```
// var x = 6;    I will NOT be executed
```



Saídas de Dados

Output

- ▶ JavaScript pode exibir dados da seguinte forma:
 - ▶ Dentro de um elemento HTML – `innerHTML`;
 - ▶ Na saída HTML – `document.write()`;
 - ▶ Em uma caixa de alerta – `window.alert()`;
 - ▶ No console do browser – `console.log()`.

innerHTML

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = 5 + 6;
```

```
</script>
```

```
</body>
```

document.write()

- ▶ Apenas deve ser utilizado para teste.

```
<h1>My First Web Page</h1>  
<p>My first paragraph.</p>  
  
<script>  
document.write(5 + 6);  
</script>
```

window.alert()

```
<h1>My First Web Page</h1>
```

```
<p>My first paragraph.</p>
```

```
<script>
```

```
window.alert(5 + 6);
```

```
</script>
```

console.log()

- Utilizado para **debugging**.

```
<script>  
console.log(5 + 6);  
</script>
```



Operadores

Operador de Atribuição e Adição

```
var x = 5;  
var y = 2;  
var z = x + y;
```

Atribuição

Adição

Operador de Multiplicação

```
var x = 5;  
var y = 2;  
var z = x * y;
```

Operadores Aritméticos

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (<u>ES2016</u>)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Operadores de Comparação

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Operadores Lógicos

Operator	Description
&&	logical and
	logical or
!	logical not



Tipos de Dados

Tipos Dinâmicos em JS

- ▶ Em JavaScript, as variáveis podem armazenar diferentes tipos de dados.

```
var x;           // Now x is undefined  
x = 5;           // Now x is a Number  
x = "John";      // Now x is a String
```

JavaScript - String

- ▶ Uma String é uma série de caracteres;
- ▶ Pode-se utilizar aspas duplas ou simples.

```
var answer1 = "It's alright";  
var answer2 = "He is called 'Johnny'";  
var answer3 = 'He is called "Johnny"';
```

JavaScript - Number

- ▶ Em JS, há apenas um tipo de número, que pode ser escrito com ou sem casa decimal.

```
var x1 = 34.00;  
var x2 = 34;
```

JavaScript - Booleans

- Podem ter apenas dois valores: **true** ou **false**.

```
var x = 5;  
var y = 5;  
var z = 6;  
  
(x == y)           // Returns true  
(x == z)           // Returns false
```

JavaScript - Arrays

- ▶ São usados para armazenar múltiplos valores em uma única variável;
- ▶ São escritos com colchetes;
- ▶ Itens de um array são separados por vírgula;
- ▶ O **primeiro índice** de um array é o **zero**.

```
var cars = ["Saab", "Volvo", "BMW"];
```

JavaScript - Arrays

- ▶ A declaração pode ter múltiplas linhas.

```
var cars = [  
    "Saab",  
    "Volvo",  
    "BMW"  
];
```

JS – Array usando new

- ▶ A declaração pode ter múltiplas linhas.

```
var cars = new Array("Saab", "Volvo", "BMW");
```

- ▶ Esta abordagem apresenta pior velocidade de execução, pois trata a variável como um objeto.

Array - Acessando os Elementos

- ▶ O acesso é feito através de um índice;
- ▶ O índice inicial é o zero.

```
var cars = ["Saab", "Volvo", "BMW"];  
document.getElementById("demo").innerHTML = cars[0];
```

JavaScript - Objetos

- Representam objetos, propriedades e métodos da vida real.

Object	Properties	Methods
	<code>car.name = Fiat</code>	<code>car.start()</code>
	<code>car.model = 500</code>	<code>car.drive()</code>
	<code>car.weight = 850kg</code>	<code>car.brake()</code>
	<code>car.color = white</code>	<code>car.stop()</code>

JavaScript – Criação de Objetos

- Pode-se utilizar as duas abordagens abaixo:

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

```
var person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

Objeto - Acessando as Propriedades

- ▶ É possível acessar as propriedades de duas formas:

```
objectName.propertyName
```

ou

```
objectName["propertyName"]
```

Objeto - Métodos

- São ações que podem ser executadas pelo objeto.

```
var person = {  
  firstName: "John",  
  lastName : "Doe",  
  id       : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

Objeto - Acessando os Métodos

- ▶ É possível acessar os métodos de duas formas:

```
objectName.methodName()
```

ou

```
name = person.fullName();
```

JavaScript - Undefined

- ▶ Variáveis sem valor, possuem valor e tipo indefinidos.

```
var car;    // Value is undefined, type is undefined
```



DOM (Document Object Model)

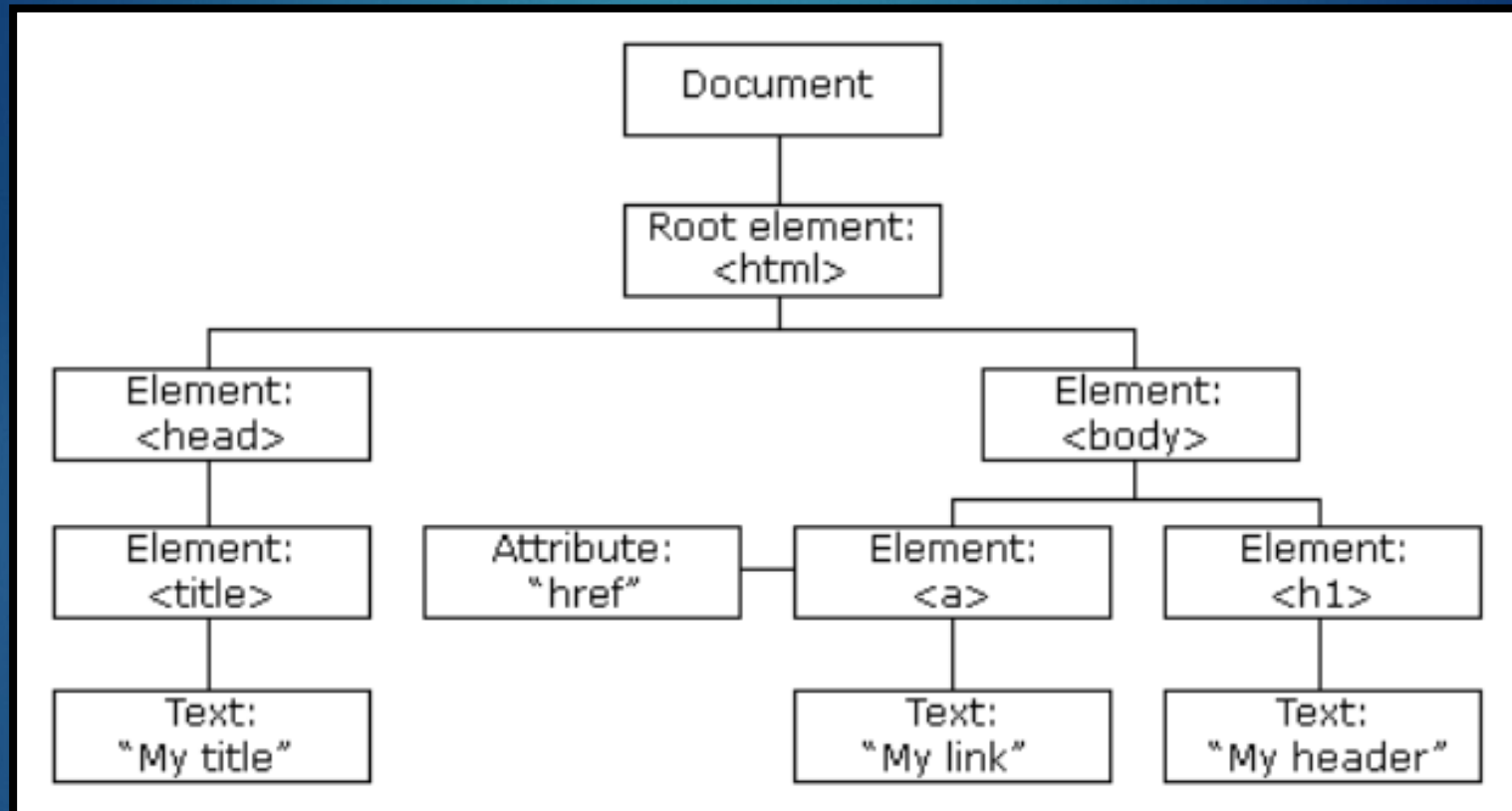
O QUE É DOM?

- ▶ Quando a página é carregada, o navegador cria um DOM da página;
- ▶ Define um padrão para acessar documentos (W3C);
- ▶ Separado em 3 partes:
 - ▶ Core DOM – Todos os tipos de documentos;
 - ▶ XML DOM – Documentos XML;
 - ▶ HTML DOM – Documentos HTML.

O QUE É HTML DOM?

- ▶ Modelo de objeto padrão e interface de programação para HTML;
- ▶ Meio de pegar, alterar, adicionar e deletar elementos HTML.

Organizado em Árvore



Métodos HTML DOM

- ▶ Métodos são ações que podem ser executadas sobre elementos HTML;
- ▶ Propriedades são valores que podem ser atribuídos ou alterados;
- ▶ A interface de programação consiste nos métodos e propriedades de cada objeto.

Métodos HTML DOM

- ▶ **getElementById** – método mais comum para acessar um elemento HTML;
- ▶ **innerHTML** – propriedade para pegar ou alterar o conteúdo de um elemento.

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = "Hello World!";
```

```
</script>
```

Objeto Documento

- ▶ Representa a página web;
- ▶ Dono de todos os objetos na página;
- ▶ Ponto de acesso a qualquer elemento HTML;

Procurar Elementos

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

Mudar Elementos

Property	Description
<i>element.innerHTML = new html content</i>	Change the inner HTML of an element
<i>element.attribute = new value</i>	Change the attribute value of an HTML element
<i>element.style.property = new style</i>	Change the style of an HTML element
Method	Description
<i>element.setAttribute(attribute, value)</i>	Change the attribute value of an HTML element

Adicionar e Deletar Elementos

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

Gerenciador de Eventos

Method

```
document.getElementById(id).onclick = function(){code}
```

Description

Adding event handler code to an onclick event



EVENTOS HTML DOM

O Que São Eventos DOM?

- ▶ Uma forma de capturar e gerenciar os diferentes tipos de eventos que podem ocorrer em um elemento HTML;
- ▶ Normalmente estão associados a uma função que será disparada assim que o evento ocorrer.

Exemplos de Eventos HTML

- ▶ Usuário clicar com o mouse;
- ▶ A página web ser carregada;
- ▶ Uma imagem ser carregada;
- ▶ O mouse passar sobre um elemento;
- ▶ Um *input* ter seu valor modificado;
- ▶ Um formulário ser enviado;
- ▶ Entre outros.

Evento click

```
<button onclick="myFunction()">Click me</button>
```

- ▶ Neste caso, quando o evento click ocorrer a função “myFunction()” será executada.

Evento onload

- ▶ Disparado quando o usuário entra em uma página web;
- ▶ Pode ser usado para verificar o tipo e a versão do *browser* do visitante, para carregar a página apropriada;

```
<body onload="checkCookies()">
```

Evento onchange

- ▶ Frequentemente utilizado para validar *inputs* de formulários;
- ▶ Disparado quando o valor do campo muda.

```
<input type="text" id="fname" onchange="upperCase()">
```

Eventos onmouseover e onmouseout

- ▶ O primeiro pode ser usado para chamar uma função quando o mouse passa sobre um elemento HTML;
- ▶ O outro ocorre quando o mouse sai de cima de um elemento HTML.

Eventos onmouseover e onmouseout

```
<!DOCTYPE html>
<html>
<body>

<div onmouseover="mOver(this)" onmouseout="mOut(this)"
style="background-color:#D94A38;width:120px;height:20px;padding:40px;">
Mouse Over Me</div>

<script>
function mOver(obj) {
    obj.innerHTML = "Thank You"
}

function mOut(obj) {
    obj.innerHTML = "Mouse Over Me"
}
</script>

</body>
</html>
```

onmousedown e onmouseup

- ▶ O primeiro ocorre quando um botão do mouse é clicado, então, quando este botão é liberado, o evento **onmouseup** é disparado. Por fim, quando o processo de clique no mouse está completo, o evento **onclick** é disparado.

onmousedown e onmouseup

- ▶ O primeiro ocorre quando um botão do mouse é clicado, então, quando este botão é liberado, o evento **onmouseup** é disparado;
- ▶ Por fim, quando o processo de clique no mouse está completo, o evento **onclick** é disparado.

onmousedown e onmouseup

```
<div onmousedown="mDown(this)" onmouseup="mUp(this)"
style="background-color:#D94A38;width:90px;height:20px;padding:40px;">
Click Me</div>

<script>
function mDown(obj) {
  obj.style.backgroundColor = "blue";
  obj.innerHTML = "Release Me";
}

function mUp(obj) {
  obj.style.backgroundColor="#D94A38";
  obj.innerHTML="Thank You";
}
</script>
```

Outros Eventos

- ▶ Para uma lista com todos os eventos, acesse:
- ▶ https://www.w3schools.com/jsref/dom_obj_event.asp



**CONSIDERE SER MEMBRO DO CANAL NO YOUTUBE E GANHE
ACESSO A CONTEÚDOS EXCLUSIVOS!**

youtube.com/@ArnaldoJunior